Workload Divide

👤 Jingjing — MySQL Infrastructure + Schema (Foundation Owner)

Focus: Backend foundation + database design

STEP I

- Part 1: Infrastructure Extension (RDS + Terraform)
- Part 2: Database Schema Design
    - Tables, keys, indexes, constraints
    - Transaction strategy
    - Documentation (design decisions + learning)

STEP III

- Part 2: Resource Efficiency Analysis

Support

- Help Person 2 with schema-related API issues

Why balanced?

- Heavy design work but less coding/debugging later

👤 Hong — MySQL API + Testing + Monitoring

Focus: Implementation + performance validation

STEP I

- Part 3: Shopping Cart API Implementation

- Part 4: Performance Testing (150 ops → mysql_test_results.json)
- Part 6: CloudWatch Monitoring

STEP I (Docs)

- Part 5: Learning Notes (MySQL side)

STEP III

Part 3: Real-World Scenario Recommendations

Why balanced?

- Coding-heavy + testing + debugging + some writing

👤 Stella — DynamoDB Design + API + Consistency

Focus: NoSQL system design + implementation

STEP II

- Part 1: DynamoDB Table Design
- Part 2: API Implementation (same endpoints)
- Part 3: Eventual Consistency Investigation

STEP II (Docs)

- Part 5: Learning Notes (DynamoDB)

Why balanced?

- Similar workload to Person 1 + 2 combined but scoped to DynamoDB only

---

👤 Person 4 — DynamoDB Testing + Final Analysis (Data Owner)

Focus: Testing + cross-system comparison + final report

STEP II

- Part 4: Required Testing → dynamodb_test_results.json
- Part 6: CloudWatch Monitoring

STEP III

- Part 0: Combine data → combined_results.json
- Part 1: Performance Comparison Tables
- Part 4: Architecture Decisions
- Part 5: Final Reflection

Why balanced?

- Less implementation, but **heaviest analysis + report writing**

Collaboration Plan (Important)

To avoid blockers:

- **Person 1 ↔ Person 2**
    - Schema must be ready before API is finalized
- **Person 2 ↔ Person 3**
    - Keep API contract identical (critical!)

- **Person 2 & 4**
    - Ensure MySQL test format matches DynamoDB
- **Person 3 & 4**
    - Validate DynamoDB results consistency

STEP I: MySQL Integration for Relational Data

(Jingjing) Part 2: Database Schema Implementation

Your Design Decisions:

1. Schema Design: How many tables? What fields in each?

   2 tables: shopping_carts and cart_items.

   Shopping_cart:

Field	Type	Purpose
cart_id	BIGINT	Primary Key. Unique ID for the cart, auto-incremented.
customer_id	BIGINT	Identifies which user owns this cart.
status	ENUM	Current state: 'active' (shopping) or 'checked_out' (completed).
created_at	DATETIME	When the cart was first created (millisecond precision).
updated_at	DATETIME	Automatically updates whenever any field in this row changes.

   Cart_item:

Field	Type	Purpose
item_id	BIGINT	Primary Key. Unique ID for this specific row/entry.
cart_id	BIGINT	Foreign Key. Connects this item to a specific shopping_cart.
product_id	BIGINT	Identifies which product was added.

quantity	INT	How many units of the product are in the cart.
created_at	DATETIME	When the item was added to the cart.
updated_at	DATETIME	When the quantity or item details were last modified.

2. Key Strategy: Primary keys, foreign keys, and constraints
   (1) Primary Keys
      (a) shopping_carts.cart_id: Uniquely identifies one specific cart session.
      (b) cart_items.item_id: Uniquely identifies one specific line item.
   (2) Foreign Keys
      (a) cart_items.cart_id is the Foreign Key. It points directly back to shopping_carts.cart_id, telling the database exactly which parent cart that specific product belongs to.
   (3) Constraint:

      `UNIQUE KEY uk_cart_product (cart_id, product_id)` — Ensures one row per product per cart. This enables `INSERT ... ON DUPLICATE KEY UPDATE quantity = quantity + VALUES(quantity)` for the `POST /shopping-carts/{id}/items` endpoint, handling both "add new item" and "update existing item" in a single atomic operation.

3. Index Strategy: Which columns need indexes for your access patterns?

index	colums	purpose
PRIMARY KEY on shopping_carts	(cart_id)	Fast cart retrieval by ID (GET /shopping-carts/{id}) — single row lookup, <50ms
idx_carts_customer_created	(customer_id, created_at DESC)	Customer purchase history queries — covers both filter and sort without filesort
PRIMARY KEY on cart_items	(item_id)	Row-level identification
uk_cart_product	(cart_id, product_id)	1) Prevents duplicate products in a cart. 2) Serves as the lookup index for JOIN when retrieving cart items by cart_id — the leftmost column matches the JOIN condition

4. Transaction Design: How to handle concurrent cart modifications?

(1) Create cart: Single INSERT, no transaction needed

(2) Add/update item: INSERT ... ON DUPLICATE KEY UPDATE — single atomic statement, handles concurrent modifications to different products in the same cart without conflicts
(3) Get cart: SELECT ... JOIN — read-only, no transaction needed
(4) For concurrent access to the same product in the same cart, InnoDB's row-level locking on the unique key ensures correctness

5. How you handle the cart-item relationship
(1) One-to-many: one cart has many items
(2) Enforced by foreign key constraint
(3) ON DELETE CASCADE ensures data integrity — no orphaned items
(4) The unique key (cart_id, product_id) models the business rule that each product appears at most once per cart (quantity is incremented instead of creating duplicate rows)

6. Any trade-offs you considered
(1) Two tables vs. single table with JSON items: More JOINs, but better data integrity, indexing, and per-item operations
(2) AUTO_INCREMENT vs. UUID: Sequential IDs are faster for InnoDB inserts but expose ordering info; acceptable for internal cart IDs
(3) BIGINT UNSIGNED vs. INT: Over-provisioned for a homework assignment, but matches production practice and avoids future migration
(4) No unit_price in cart_items: Matches the OpenAPI spec (only product_id + quantity); price lookup would be handled by a separate product service if needed

(Hong)Part 3: Shopping Cart API Implementation

- **Did your first schema design meet performance requirements?**

  Yes. The original schema from schema.sql required no modifications. All 150 operations completed in ~11 seconds (well under the 5-minute limit) with a 100% success rate.

- **What queries were slower than expected?**

  The very first create_cart request took 338.51 ms — nearly 5x the average of 75.57 ms. This wasn't a query issue but a cold-start penalty.

add_items was slightly slower than get_cart (avg 76.85 ms vs 67.74 ms) because it involves a transaction with two operations (status check + upsert) plus write overhead (InnoDB locking and WAL flush), while get_cart is read-only.

- **How did you optimize for the 150-operation test?**

  1. Connection pooling — MaxOpenConns(25) and MaxIdleConns(5) to keep connections warm between requests and support concurrent access.

  2. Upsert with ON DUPLICATE KEY UPDATE — reduces add-item to one DB round-trip instead of SELECT + conditional INSERT/UPDATE.

  3. Minimal transaction scope — the transaction in addCartItem only wraps the status check + upsert. The post-insert SELECT to return the response runs outside the transaction to minimize lock duration.

  4. Parameterized queries (? placeholders) — prevents SQL injection and allows MySQL to cache query plans for repeated operations.

(Hong)Part 5: Learning Notes

Implementation Journey

Note your problem-solving process:

- What would you do differently next time?

  Add a connection warm-up on startup: ping the DB a few times after init to pre-establish idle connections, avoiding the cold-start latency spike on the first real request.

Results:

---

create_cart: count=50, success=50, avg=75.57ms, min=61.15ms, max=338.51ms

add_items: count=50, success=50, avg=76.85ms, min=62.89ms, max=306.48ms

get_cart: count=50, success=50, avg=67.74ms, min=62.45ms, max=89.81ms

---

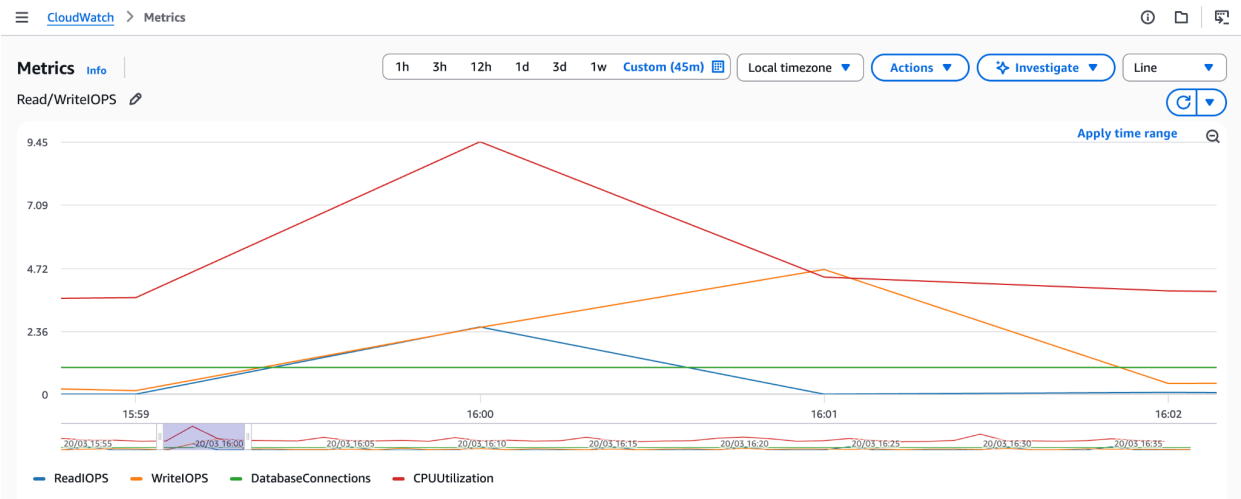
Overall: count=150, success=150/150, avg=73.39ms, total_time=11.01s

(Hong)Part 6: CloudWatch Monitoring

Key Metrics to find:

- **CPU Utilization:** Look for the `CPUUtilization` graph. (Shows if your DB is sweating during the Flash Sale).
- **Connections:** Look for `DatabaseConnections`. (Crucial for Phase 1 to see if you hit a "Too many connections" error).
- **I/O Performance:** Look for `ReadIOPS` and `WriteIOPS`.
- **Target response time** and **FreeableMemory**

Observation



CPUUtilization (red)

- Baseline: ~4% idle

- Peak: ~9.45% at 16:00 during the test

- Drops back to ~4% after test completes

- Very light load — the db.t3.micro was barely stressed

ReadIOPS (blue)

- Peak: ~2.36/sec around 16:00

- From the SELECT queries: fetching carts after creation, fetching items after upsert, and the 50 GET requests

- Drops to ~0 after the test

WriteIOPS (orange)

- Peak: ~2.36/sec, slightly lagging behind reads

- A second smaller spike at ~16:01 (~4.72), likely MySQL flushing InnoDB buffer pool pages to disk (background write-back)

- From the 50 INSERTs + 50 UPSERTs + index updates

DatabaseConnections (green)

- Steady at 1 throughout the entire test

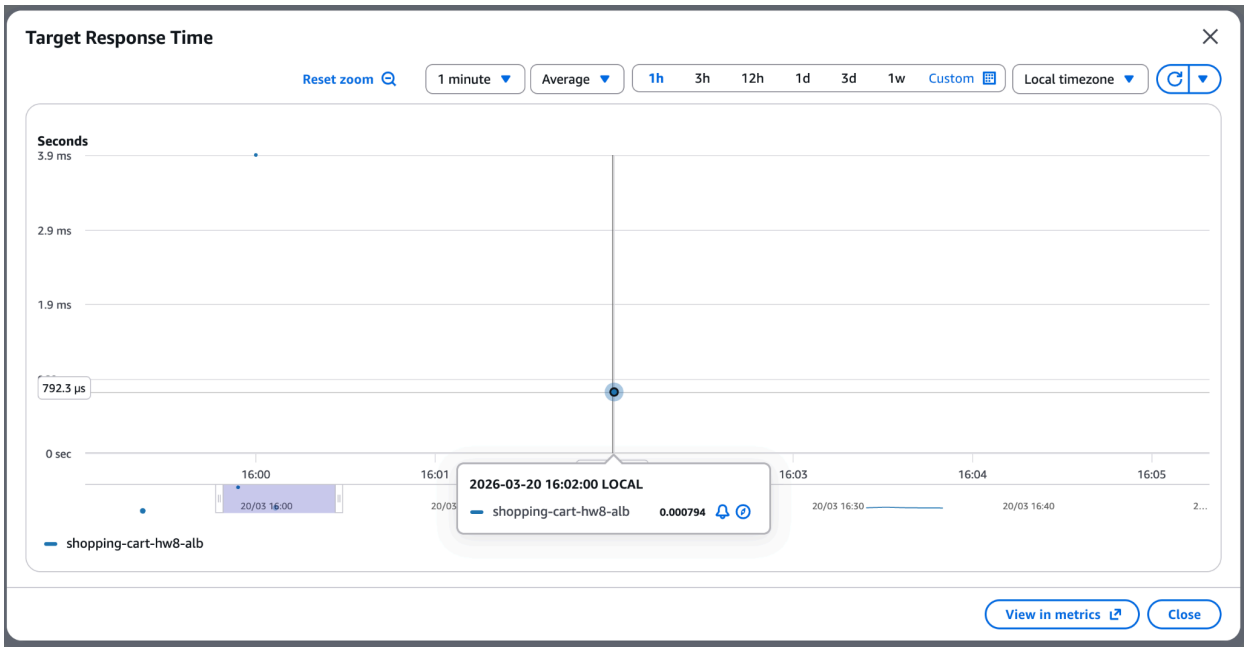
- Confirms the sequential nature of your test — only 1 connection was ever needed at a time

Target Response Time

- Peak: ~3.9 ms at 16:00 — the first requests when the test started (likely includes connection warm-up)

- Dropped to: ~969 microseconds (~1 ms) by 16:02 — after the connection pool is warm

- This is excellent. Sub-4ms response times mean your API + DB queries are very fast



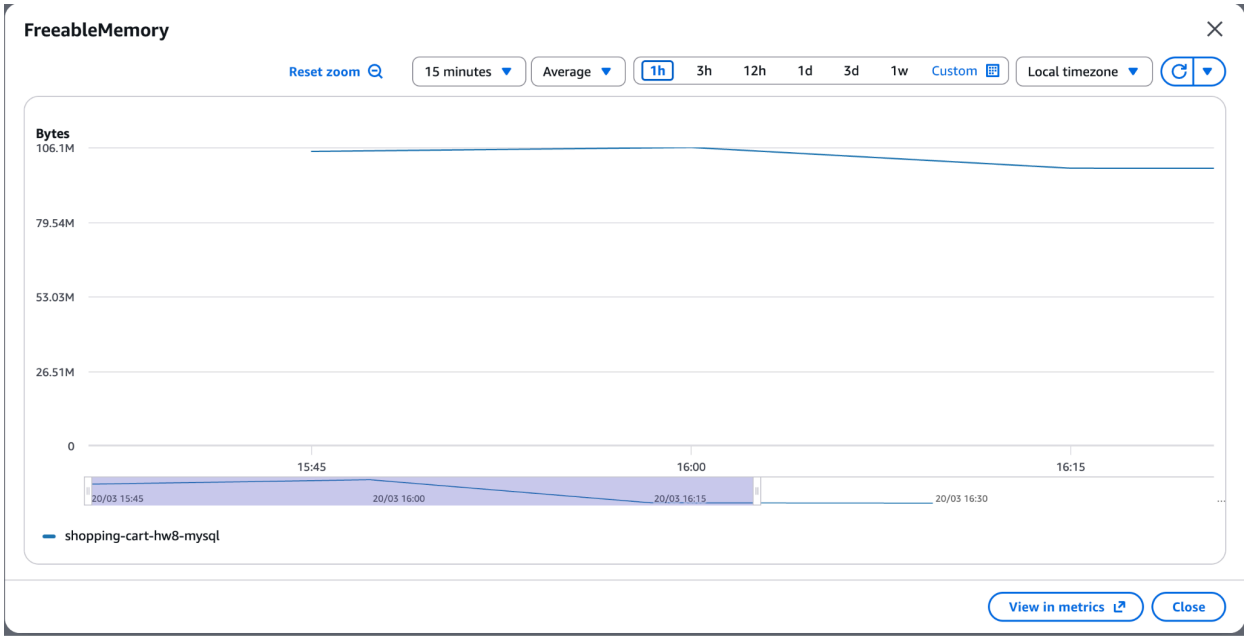
FreeableMemory

- Steady at ~106 MB throughout the entire test

- Barely moved — the test had no noticeable impact on memory

- db.t3.micro has ~1 GB total, so ~106 MB free is normal (MySQL's InnoDB buffer pool and other internals use the rest at baseline)

- No memory pressure at all for this workload



STEP II: DynamoDB Integration instead of NoSQL

(SZ)Part 1: DynamoDB Table Design Challenge

Your Design Decisions:

a. *Partition Key Strategy: What key ensures even distribution?*
   - I chose a **composite primary key** design where the partition key is pk, and for each cart the value is stored as CART#<cart_id>. The cart_id is generated uniquely for each cart, which helps distribute requests evenly across DynamoDB partitions under normal shopping load. Since most application operations are scoped to one cart at a time, using cart ID as the partition key also aligns naturally with the main access pattern.

b. *Sort Key Need: Do you need a sort key for any operations?*
   - **Yes.** I use a sort key sk to distinguish different item types stored under the same cart partition. For example:
     sk = META stores cart metadata
     sk = ITEM#<product_id> stores each cart item
     This allows all data for one cart to be grouped together in a single partition while still supporting multiple rows per cart. The sort key is important because it lets me

use a single Query operation to retrieve the full cart, including both cart metadata and all cart items.

c. *Table Structure: Single table vs multiple tables?*
   - I chose a **single-table** design. Instead of creating separate DynamoDB tables for carts and cart items, I store both cart metadata and cart item rows in the same table. This follows DynamoDB's access-pattern-driven design style and makes cart retrieval efficient, since all related data is colocated under one partition key.

d. *Attribute Design: How to handle cart items (embedded vs separate)?*
   - I chose to store cart items as **separate rows**, not as an embedded list inside the cart metadata item. Each cart item is stored as its own record with:
     - the same partition key as the cart
     - a sort key like ITEM#<product_id>

     This design makes add/update item operations simpler and more efficient because the application can update one cart item row without rewriting the entire cart document. It also better mirrors the MySQL logic where each product appears at most once per cart.

e. *Index Strategy: Any secondary indexes needed?*
   - For the required API endpoints, no additional secondary index is strictly necessary. The main operations are:
     - create cart
     - get cart by cart ID
     - add/update item in cart

     All of these are fully supported by the primary key design.

Document Your Approach:

a. Why you chose your partition key
   - I chose cart ID as the logical partition key because the dominant access pattern is retrieving or updating a single cart at a time. Cart IDs are unique, which helps distribute traffic evenly across partitions and avoids hot partition problems under normal shopping activity. This design also keeps all rows related to one cart together, which is important for efficient reads.

b. How you handle the cart-item relationship
   - I handle the cart-item relationship by storing both cart metadata and cart items in the same table under the same partition key. The cart metadata row uses sk = META, while each cart item uses a sort key like ITEM#<product_id>. This models a one-to-many relationship without joins. To retrieve a cart, the application queries all rows for that cart partition and reconstructs the final cart response in application code.

c. How your design compares to the MySQL approach from STEP I
    - The MySQL implementation uses a normalized relational model with two tables: one for shopping carts and one for cart items, connected through a foreign key. That design relies on joins and relational constraints to maintain cart-item relationships.

      The DynamoDB design is more access-pattern-driven. Instead of multiple tables and joins, all data for one cart is stored together in one table under a shared partition key. This reduces the need for joins and fits DynamoDB's strengths, but it moves more responsibility to the application layer for reconstructing responses and managing update behavior.

d. Any trade-offs you identified
    - One trade-off is that DynamoDB simplifies the physical data access pattern for cart retrieval, but it requires more careful upfront design around keys and item structure. Another trade-off is that DynamoDB does not provide the same relational guarantees as MySQL, such as foreign keys and join-based consistency. In return, it offers a flexible schema and a design that can scale well for high-volume key-based access patterns. I also considered embedding all items inside a single cart record, but I chose separate item rows because that makes item-level updates cleaner and avoids rewriting the entire cart on every change

(SZ)Part 2: API Implementation

Investigation Questions:

- *How does DynamoDB's attribute value format affect your application data structures?*

  DynamoDB stores data as typed attribute-value pairs rather than rows in a relational schema, so the application needs to map Go structs into DynamoDB items and back again.

  In my implementation, this means defining explicit struct types for cart metadata and cart item rows, then using marshaling and unmarshaling with the AWS SDK. It also influenced the data model because I had to represent the shopping cart as multiple DynamoDB items with pk and sk attributes instead of relying on relational joins. Compared with MySQL, more of the data reconstruction logic moves into the application layer.

- *What DynamoDB operations (PutItem, GetItem, Query) are most appropriate for each endpoint?*

  For POST /shopping-carts, the most appropriate operation is PutItem because creating a new cart writes a single metadata row with pk = CART#<cart_id> and sk = META.

  For GET /shopping-carts/{id}, the best operation is Query because the full cart is stored as multiple rows under the same partition key, so querying by pk returns both the metadata row and all item rows.

  For POST /shopping-carts/{id}/items, the implementation uses a combination of operations: GetItem to verify the cart metadata exists, GetItem again to check whether that product already exists in the cart, PutItem to insert or overwrite the item row, and UpdateItem to update the cart metadata's updated_at timestamp. This matches the single-table design where related entities are grouped by partition key rather than joined across tables.

- *How do you handle eventual consistency in your operations?*

  By default, DynamoDB reads are eventually consistent, so recently written data may not always appear immediately in a following read.

  In my implementation, I handle this by using **strongly consistent reads in** places where correctness matters most during write workflows, such as verifying the cart metadata exists before adding an item. For cart retrieval, I support the default eventually consistent read path, but I also added an option to request a strongly consistent read using ?consistent=true for testing and investigation. This makes it possible to compare the behavior of eventual and strong consistency directly and observe how DynamoDB consistency affects read-after-write behavior.

(SZ)Part 3: Eventual Consistency Investigation

Investigation Questions:

- *How frequently do you observe eventual consistency delays?*

  I manually tested immediate read-after-write behavior after cart creation and item insertion using both the default eventually consistent read path and a strongly

consistent read path. In these **small-scale tests**, I did **not observe noticeable consistency delays.**

In **20 trials** for create-then-immediate-read and 20 trials for add-item-then-immediate-read, I did **not observe visible eventual consistency delays**. Both the default read path and the strongly consistent read path returned the latest expected state in all tested runs.

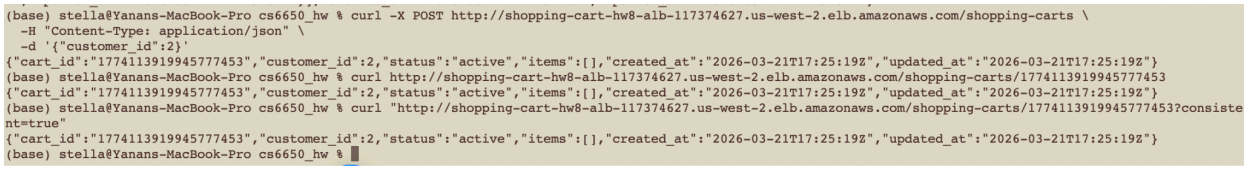
- *What application patterns are most affected by consistency delays?*

  The application patterns most affected by consistency delays are **read-after-write interactions**, especially when users expect to immediately see the result of their action. In a shopping cart workflow, this includes creating a cart and then immediately loading it, or adding an item and expecting the updated cart contents to appear right away. If eventual consistency caused stale reads, the user might think the cart was not created successfully or that the item was not added, even though the write had already succeeded. Rapid repeated updates to the same cart would also be more sensitive than simple independent reads.

- How can you design your application to handle consistency gracefully?

  In my implementation, I added support for **?consistent=true** so I could explicitly request a strongly consistent read for testing and correctness-sensitive cases. Another way to handle consistency gracefully is to design the application and UI to tolerate short propagation delays, such as **retrying** a read after a write or showing optimistic updates in the client. For shopping cart operations, using **strong reads immediately** after cart creation or item updates can improve user confidence when the latest state must be reflected right away.

Screenshot: (Manual)No noticeable eventual consistency delays on creating a new cart.



Screenshot: (Manual)No noticeable eventual consistency delays on adding item.

```
(base) stella@Yanans-MacBook-Pro cs6650_hw % curl -X POST http://shopping-cart-hw8-alb-117374627.us-west-2.elb.amazonaws.com/shopping-carts/1774113919945777453/i
tems \
  -H "Content-Type: application/json" \
  -d '{"product_id":200,"quantity":1}'
{"item_id":"item_200","product_id":200,"quantity":1,"created_at":"2026-03-21T17:31:30Z","updated_at":"2026-03-21T17:31:30Z"}
(base) stella@Yanans-MacBook-Pro cs6650_hw % curl http://shopping-cart-hw8-alb-117374627.us-west-2.elb.amazonaws.com/shopping-carts/1774113919945777453
{"cart_id":"1774113919945777453","customer_id":2,"status":"active","items":[{"item_id":"item_200","product_id":200,"quantity":1,"created_at":"2026-03-21T17:31:30
Z","updated_at":"2026-03-21T17:31:30Z"}],"created_at":"2026-03-21T17:25:19Z","updated_at":"2026-03-21T17:31:30Z"}
(base) stella@Yanans-MacBook-Pro cs6650_hw % curl "http://shopping-cart-hw8-alb-117374627.us-west-2.elb.amazonaws.com/shopping-carts/1774113919945777453?consiste
nt=true"
{"cart_id":"1774113919945777453","customer_id":2,"status":"active","items":[{"item_id":"item_200","product_id":200,"quantity":1,"created_at":"2026-03-21T17:31:30
Z","updated_at":"2026-03-21T17:31:30Z"}],"created_at":"2026-03-21T17:25:19Z","updated_at":"2026-03-21T17:31:30Z"}
(base) stella@Yanans-MacBook-Pro cs6650_hw %
```

Screenshot: (Script) 20 Trails create-then-immediate-read and add-item-then-immediate-read

```
============================================================
Consistency Test Summary
============================================================

create_then_immediate_read
------------------------------------------------------------
Trials: 20
Normal read matched expected: 20/20
Strong read matched expected: 20/20
Normal read mismatch count: 0
Strong read mismatch count: 0

add_item_then_immediate_read
------------------------------------------------------------
Trials: 20
Normal read matched expected: 20/20
Strong read matched expected: 20/20
Normal read mismatch count: 0
Strong read mismatch count: 0

Detailed results saved to: dynamodb_consistency_results.json
(base) stella@Yanans-MacBook-Pro dynamodb %
```

Part 4: Required Identical Testing

(SZ)Part 5: Learning Notes

What Surprised You?

- How different DynamoDB **design feels** compared with MySQL. In MySQL, I naturally think in terms of normalized tables, foreign keys, and joins. In DynamoDB, I have to start from the access patterns first and then shape the data model around them. That was a different mindset.

- Another thing that surprised me was that eventual consistency did not visibly affect my small-scale testing. I expected at least a few stale reads when I tested immediate read-after-write behavior, but in my manual checks and repeated consistency script, both the default read path and the **strongly consistent read path returned the latest state** in all tested runs. This was a useful reminder that eventual consistency is a property of the system design, but it does not mean stale reads will always be easy to observe in every small environment.

Document any unexpected discoveries:

- Did your partition key strategy work as expected?

  Yes. My partition key strategy worked as expected for the access patterns in this homework. I used a single-table design with:

  pk = CART#<cart_id>

  sk = META for cart metadata

  sk = ITEM#<product_id> for cart items

  This design grouped all data for one cart under the same partition key, which made cart retrieval straightforward with a single Query operation. It also allowed item-level updates without rewriting the entire cart.

- Were there NoSQL concepts that differed from your expectations?

  Yes. The biggest difference was how much more application-side modeling DynamoDB requires. In MySQL, relationships and integrity are handled more directly through schema design, constraints, and joins. In DynamoDB, I had to represent related entities explicitly through partition key and sort key design, and then reconstruct the final cart response in application code.

- How did eventual consistency affect your testing?

  In my testing, eventual consistency did not create visible problems. The main takeaway was not that eventual consistency disappears, but that it was not observable in this small-scale homework setup. Even so, shopping carts are still a read-after-write-sensitive use case, so supporting strongly consistent reads remains a good design option when immediate correctness matters.

Design Evolution

At first, I considered a **simpler design where each cart would be stored as one DynamoDB item with all cart items embedded in a list.** That design would make whole-cart retrieval easy, but it would require rewriting the full cart every time an item changed.

I **changed to a single-table design with separate META and ITEM rows under the same partition key** because it fit DynamoDB's access-pattern style better. This allowed me to:

use a sort key meaningfully

update cart items individually
retrieve the full cart with one Query
keep the design closer to DynamoDB's typical single-table modeling approach

Note your design iteration process:

- *What partition key did you try first and why did you change it?*

  My final partition key strategy was based on cart ID, and that ended up being the right choice. I did not change away from cart ID because it matched the main access pattern from the beginning: all core operations were centered around retrieving or updating one cart at a time.

- *Did you encounter hot partition issues during testing?*

  No. I did not encounter hot partition issues during testing. The workload spread requests across many unique cart IDs, so there was no obvious sign of concentrated traffic on a single partition. This was expected because cart IDs are unique and the test workload did not create extreme contention on the same cart.

- *How did you validate your design choices?*

  First, I verified that the deployed API supported the required endpoints.Second, I confirmed the data layout directly in DynamoDB by scanning the table and observing the expected META and ITEM rows. Third, I validated the consistency behavior by comparing the default read path and the strongly consistent read path in repeated immediate read-after-write tests.

Part 6: CloudWatch Monitoring

Monitor DynamoDB-specific metrics:

- Request latency and consumed capacity: DynamoDB internal latency was 1-3ms (Put ~3.3ms, Get ~1.47ms, Query ~1.6ms), with read usage peaking at ~1.92 units/sec and write usage at ~2.5 units/sec — on-demand mode absorbed the full burst.
- Throttling events : Zero throttled events for both reads and writes across the entire test.
- Partition distribution patterns: No hot partitions observed — each cart uses a unique cart ID as partition key, spreading traffic evenly across 50 distinct partitions, and no Scan operations were used (only Query and GetItem).

- Error rates and types: Zero errors across all 150 operations in every test run (100% success rate), with no DynamoDB-specific exceptions encountered.

STEP III: Database Comparison & Analysis

Part 1: Performance Comparison Table

Complete this table using data from your `combined_results.json` (Part 0):

Metric	MySQL	DynamoDB Test-1(us_west) oregon	DynamoDB Test-2(us_west) oregon	DynamoDB Test-3(us_west) oregon	DynamoDB Test-3(us_east) Virginia	Winner	Margin
Avg Response Time (ms)	73.39	224.47	223.09	222.07	103.46	MySQL	30.07ms
P50 Response Time (ms)	70.21	222.92	220.81	220.76	100.77	MySQL	32.92ms
P95 Response Time (ms)	105.32	233.40	244.90	234.76	115.88	MySQL	20.67ms
P99 Response Time (ms)	338.51	237.18	255.21	247.11	139.72	DynamoDB	166.76ms
Success Rate (%)	100	100	100	100	100	Tie	0
Total Operations	150	150	150	150	150		

Operation-Specific Breakdown

Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
CREATE_CART	75.57	102.48	26.91ms
ADD_ITEMS	76.85	112.15	35.30ms
GET_CART	67.74	99.68	31.94ms

Analysis Framework:

- **How frequently did you experience consistency delays?** Zero — across Stella's 40 consistency trials and my 150-op performance tests, both eventually consistent and strongly consistent reads returned the expected state every time.
- **What application patterns were most affected by eventual consistency?** Read-after-write patterns like creating a cart and immediately retrieving it, or adding an item and expecting it to appear instantly — though at our test scale these delays were never observable.
- **How would you design applications to handle each consistency model?** For MySQL, ACID transactions handle consistency by default; for DynamoDB, use strongly consistent reads for write-critical paths (like cart creation confirmation) and accept eventual consistency for less sensitive reads, similar to Stella's consistent=true parameter approach.

Part 2: Resource Efficiency Analysis

Scaling Analysis:

- **How do resource requirements change with load for each database?** MySQL requires manual scaling — increasing instance size (vertical) or adding read replicas, and connection pool limits (MaxOpenConns=25) create a hard ceiling; DynamoDB on-demand scales automatically with zero configuration, as our test showed 100% write throughput consumed briefly with no throttling.
- **Which approach offers more predictable resource consumption?** DynamoDB — our test data showed a P50-P99 spread of only ~39ms (Virginia)

compared to MySQL's ~239ms spread, and CloudWatch confirmed steady resource usage with no memory pressure or connection spikes.
- **What are the capacity planning implications for each technology?** MySQL requires upfront decisions about instance size, connection limits, and storage, with costly downtime for vertical scaling; DynamoDB on-demand eliminates capacity planning entirely, charging per-request, making it ideal for unpredictable workloads like flash sales where traffic can spike dramatically.

Part 3: Real-World Scenario Recommendations

Scenario A: Startup MVP (100 users/day, 1 developer, limited budget, quick launch)
Your Recommendation: MySQL

Key Evidence: At 100 users/day, DynamoDB's on-demand pricing would be pennies, but so is a db.t3.micro at ~$15/month. The cost difference is negligible, but MySQL gives you relational features for free — things you'd have to handle manually in application code with DynamoDB.

Scenario B: Growing Business (10K users/day, 5 developers, moderate budget, feature expansion)
Your Recommendation: MySQL with DynamoDB for specific use cases

Key Evidence: MySQL's ACID transactions and schema enforcement (avg 73.39ms, 3x faster than DynamoDB's 222ms) protect data integrity, while DynamoDB can handle simple high-throughput needs like session storage.

Scenario C: High-Traffic Events (50K normal, 1M spike users, revenue-critical, can invest in infrastructure)
Your Recommendation: DynamoDB

Key Evidence: DynamoDB auto-scales for 20x traffic spikes with no configuration and showed a tighter P50-P99 spread (~39ms vs MySQL's ~239ms), while MySQL's fixed connection pool would bottleneck under 1M concurrent users.

Scenario D: Global Platform (millions of users, multi-region, 24/7 availability, enterprise requirements)
Your Recommendation: DynamoDB (Global Tables)

Key Evidence: Our cross-region test showed DynamoDB at 103.46ms (Virginia) vs 222ms (Oregon), proving region proximity matters, while MySQL requires manual replica setup and complex failover management.

Part 4: Your Evidence-Based Architecture Recommendations

1. Shopping Cart Winner: DynamoDB — MySQL was ~30ms faster on average (73ms vs 103ms), but DynamoDB's P99 was 166ms better (139ms vs 306ms), delivering more predictable performance where it matters most.

2. Supporting Evidence: Average: MySQL wins by 30.07ms. P99: DynamoDB wins by 166.76ms. DynamoDB needed more upfront design (single-table modeling, pk/sk strategy) but required zero capacity tuning — no connection pools, no instance sizing, O(1) lookups at any scale.

3. When to Choose MySQL: When you need complex JOINs, ad-hoc queries, strict ACID transactions, or the schema is still evolving — MySQL is more forgiving than DynamoDB's rigid key design.

4. Polyglot Strategy: Shopping carts → DynamoDB (key-based access, auto-scaling). User sessions → DynamoDB (key-value + TTL expiration). Product catalog → MySQL (relationships, search, ad-hoc queries). Order history → DynamoDB (append-heavy, customer ID partition key, infinite scale).

Part 5: Learning Reflection

What Surprised You?

Results were unexpected considering the time complexity

	Dynamo DB	MySQL
Create()	O(1)	O(logn)

Add()	O(1)	O(logn)
get()	O(1+k) (k is number of items)	O(logn+k)

dynamoDB was supposed to win theoretically.

Factors leading to the loss could be due to

Latency: Virginia servers not as available as Oregon servers?

MySQL's Go driver sends queries over a single persistent TCP connection (1 round-trip per operation), while DynamoDB's AWS SDK makes separate HTTPS API calls for each operation (up to 4 round-trips for add_items), so the ~30ms average gap comes from network overhead in the application layer, not database performance — both responded in 1-4ms internally.

What Failed Initially?

Schema: Embedding cart items as a list inside one record didn't scale — switched to separate META/ITEM rows so individual items could be updated without rewriting the entire cart

Performance: DynamoDB seemed slow at ~222ms from Oregon — redeploying to Virginia dropped it to ~103ms, proving it was network distance, not the database

Testing: Had to run 3 tests in Oregon then deploy fresh infra in Virginia to isolate network latency as the variable

Configuration: ECS crashed on wrong image tag (`:v2` vs `:latest`), and Dockerfile needed to be built from repo root, not `dynamodb/`

Key Insights Gained

1.Benchmarks measure the whole stack, not just the database — both DBs responded in 1-4ms internally, the ~30ms gap was all application/network layer

2.O(1) vs O(log n) doesn't matter at 150 operations — it matters at millions

3.DynamoDB wins on predictability (P99), MySQL wins on average speed

4.Client-to-region distance amplifies DynamoDB's latency more than MySQL's due to multiple HTTPS round-trips vs single TCP connection

5.The "better" database depends on what you optimize for — there's no universal winner